Relatório — Tarefa 1_A (TEG): Grafo da base Iris

Autores: João Arthur Lima da Silva e Rebecca Reinert Gesser · **Disciplina:** Teoria dos Grafos (TEG), UDESC, 2026/2 · 8 de outubro de 2026

1. Introdução

Este trabalho implementa, em linguagem C, um grafo não direcionado como tipo abstrato de dados (TAD), representado por uma matriz de adjacências montada a partir da base *Iris dataset*. Cada uma das 150 observações vira um vértice, e dois vértices são ligados por uma aresta quando a distância euclidiana normalizada (DEN) entre suas medidas é menor ou igual a 0,3.

Além da carga primária, o sistema persiste o grafo em um arquivo CSV com um cabeçalho de métricas, recarrega esse arquivo sem reprocessar a base original e oferece um script Python para visualizar o grafo em 3D com rotação pelo mouse.

2. Base de dados e carga primária

A base `original_IrisDataset.csv` tem 150 observações (50 por espécie) e cinco colunas: `variety`, `sepal.length`, `sepal.width`, `petal.length` e `petal.width`, com medidas em centímetros.

- **Coluna ignorada:** a primeira coluna (`variety`, nome da espécie) é descartada na leitura pelo formato `%*[^,]` do `sscanf`, como pede o requisito b. O grafo usa só as quatro medidas.
- **Vértices:** a i-ésima observação do arquivo vira o vértice c*i*, então V = {c1, c2, …, c150} (requisito c).
- **Tamanho não fixo:** o número de observações não está escrito no código. O vetor de medidas cresce com `realloc` a cada linha lida, e a matriz é alocada depois, com o total lido.
- **Formato dos números:** a base grava valores como `.2`, e o `%lf` do `sscanf` aceita essa forma. Linhas em branco são ignoradas; uma linha inválida interrompe a carga com mensagem de erro.

A base tem uma observação repetida: c102 e c143 têm as mesmas medidas (5,8; 2,7; 5,1; 1,9). Por isso a menor distância entre vértices distintos é exatamente 0.

3. Metodologia: DE, DEN e limiar

Com a normalização min-max, o limiar DEN ≤ 0,3 equivale a uma distância euclidiana de até 2,1256 cm entre duas flores. A carga primária segue os quatro passos da especificação:

1. **Tabela de DEs.** Para todo par de vértices (vi, vj) com i < j, calcula-se a distância euclidiana nas quatro medidas. A tabela é simétrica, então basta a parte acima da diagonal.
2. **Tabela de DENs.** A própria tabela de DEs é normalizada para [0, 1] pela normalização min-max. O mínimo e o máximo são tomados sobre os pares com i < j; a diagonal (distância de um vértice para ele mesmo) fica de fora.
3. **Limiar.** Existe aresta (vi, vj) se e somente se DEN(vi, vj) ≤ 0,3.
4. **Matriz de adjacências.** Só os pares com i < j são avaliados e o valor é copiado para [j][i]. A diagonal fica em 0 (sem laços) e cada par gera no máximo uma aresta (sem arestas múltiplas).

DE(vi, vj) = $\sqrt{\left[\ \Sigma\ (xik - xjk)^2\ \right]}$, somando k = 1 a 4 (as quatro medidas de cada flor)

DEN(vi, vj) = (DE(vi, vj) − DEmin) / (DEmax − DEmin)

Na base usada, DEmin = 0 (par c102–c143) e DEmax = 7,085196 cm (par c14–c119). Logo DEN = DE / 7,085196, e o limiar 0,3 corresponde a DE ≤ 0,3 × 7,085196 = 2,1256 cm. Quando dois pares empatam em um extremo, vale o primeiro encontrado na varredura por linhas. Se todas as DEs forem iguais (uma base com só 2 observações, por exemplo), a divisão por zero é evitada: todas as DEN ficam 0.

4. Implementação em C

O grafo é um TAD opaco: `grafo.h` expõe só o tipo `Grafo` e as funções, e os campos da struct ficam visíveis apenas em `grafo.c`. O código segue o padrão C99 e compila sem avisos com `-Wall -Wextra -pedantic`.

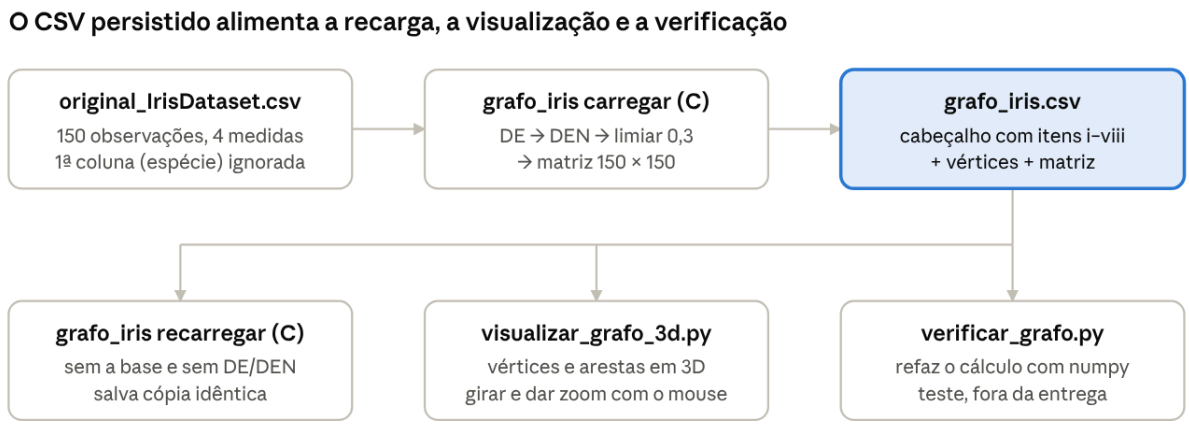


Diagrama: fluxo entre os programas · a carga primária gera o CSV usado pelos outros três

Só a carga primária lê a base e calcula DE e DEN; a recarga e a visualização partem apenas do arquivo `grafo_iris.csv`. O script de verificação é um teste: refaz o cálculo a partir da base com numpy e compara com esse arquivo.

Arquivo	Papel
`grafo.h`	Interface do TAD: tipo opaco `Grafo` e protótipos
`grafo.c`	Carga primária, persistência, recarga e consultas
`main.c`	Programa de linha de comando com os modos `carregar` e `recarregar`
`Makefile`	Compilação com gcc (`mingw32-make` no Windows)
`visualizar_grafo_3d.py`	Visualização 3D interativa do grafo salvo

```
typedef struct {
    double m[4];   /* sepal.length, sepal.width, petal.length, petal.width */
} Flor;

typedef struct {
    double valor;
    int i, j;      /* par de vertices que gerou o valor */
} Extremo;

struct grafo {
    int n;
    int **adj;     /* matriz de adjacencias n x n */
    Flor *flores;  /* medidas de cada vertice */
    Extremo maior_de, menor_de, maior_den, menor_den;
};
```

Função	O que faz
`grafo_carregar_base`	Lê a base, monta a tabela de DEs, normaliza para DENs, aplica o limiar e cria a matriz
`grafo_salvar`	Grava o CSV com cabeçalho, vértices e matriz
`grafo_carregar_salvo`	Reconstrói o grafo a partir do CSV salvo, sem a base original
`grafo_liberar`	Libera a memória do grafo
`grafo_imprimir_cabecalho`	Escreve as linhas i a viii, na tela ou no arquivo
`grafo_grau`	Grau de um vértice (um laço conta 2)
`grafo_num_arestas`	Número de arestas, contando laços e arestas múltiplas
`grafo_componentes`	Conta os componentes conexos por busca em profundidade (DFS) e devolve os tamanhos

Decisões de implementação:

- **Matriz `int **`.** Cada linha é alocada com `calloc`. As entradas são inteiras, não só 0/1, para que laços e arestas múltiplas possam ser contados em qualquer grafo, não só no gerado aqui.
- **Uma tabela só.** A tabela de DEs é normalizada no próprio lugar e vira a tabela de DENs. Ela existe só durante a carga primária; o grafo guarda apenas a matriz e os extremos exigidos no cabeçalho.
- **Um cabeçalho para tela e arquivo.** `grafo_imprimir_cabecalho` escreve as mesmas linhas no terminal e no CSV, então o que aparece na tela é exatamente o que foi salvo.
- **Grau.** É a soma da linha do vértice; o laço na diagonal é somado de novo, pois conta 2.
- **Componentes conexos.** Uma DFS recursiva parte de cada vértice ainda não visitado e devolve quantos vértices alcançou, que é o tamanho do componente.

5. Persistência em CSV e recarga

O grafo é salvo em `grafo_iris.csv`, e a recarga reproduz o arquivo byte a byte: salvar, recarregar e salvar de novo gera um arquivo idêntico (mesmo hash SHA-256). Cada linha começa por uma chave que diz o que ela contém.

Item	Linha do CSV	Campos após a chave
i	`total_vertices`	Total de vértices lidos
ii	`maior_DE`	Valor, vértice, vértice
iii	`menor_DE`	Valor, vértice, vértice
iv	`maior_DEN`	Valor, vértice, vértice
v	`menor_DEN`	Valor, vértice, vértice
vi	`grau_maximo`, `grau_minimo`	Grau
vii	`tipo_grafo`	`simples` ou `multigrafo`, `lacos`, quantidade, `arestas_multiplas`, quantidade
viii	`componentes_conexos`, `tamanhos_componentes`	Quantidade de componentes; tamanho de cada um
extra	`total_arestas`	Número de arestas

Depois do cabeçalho vêm duas seções:

- `vertices`: uma linha de título com os nomes das medidas e 150 linhas no formato `cI,m1,m2,m3,m4`. As medidas são dados do vértice e servem para posicioná-lo na visualização 3D.
- `matriz_adjacencia`: uma linha de título `matriz_adjacencia,c1,…,c150` e 150 linhas `cI,a1,…,a150` com 0 ou 1.

Na recarga (`grafo_carregar_salvo`), o arquivo é lido na mesma ordem em que foi gravado. Primeiro vem `total_vertices`, que define o tamanho da matriz a alocar, e depois os quatro extremos de DE e DEN, lidos sem recalcular nenhuma tabela. As linhas de grau, tipo e componentes são puladas, porque saem da própria matriz; em seguida vêm as medidas e a matriz. A base original não é aberta, e um arquivo incompleto ou com valor inválido é recusado com mensagem de erro.

6. Resultados

Com limiar 0,3, o grafo é simples, conexo e denso: 5.250 arestas entre 150 vértices, 47,0% dos 11.175 pares possíveis, com grau médio 70. Os valores abaixo são os do cabeçalho de `grafo_iris.csv`.

Item do cabeçalho	Resultado
i. Total de vértices	150
ii. Maior DE	7,085196 cm, entre c14 e c119
iii. Menor DE	0,000000 cm, entre c102 e c143
iv. Maior DEN	1,000000, entre c14 e c119

Item do cabeçalho	Resultado
v. Menor DEN	0,000000, entre c102 e c143
vi. Grau máximo / mínimo	93 (vértice c78) / 30 (vértice c119)
vii. Simples ou multigrafo	Simples: 0 laços e 0 arestas múltiplas
viii. Componentes conexos	1 componente, com 150 vértices

- **Extremos de DEN = extremos de DE.** A normalização min-max preserva a ordem das distâncias, então o par mais distante tem DEN 1 e o mais próximo tem DEN 0. O par mais distante liga a menor Setosa, c14 (4,3; 3,0; 1,1; 0,1), à maior Virginica, c119 (7,7; 2,6; 6,9; 2,3).
- **Graus.** c119 é um ponto extremo da base e tem só 30 vizinhos a até 2,13 cm. O grau máximo, 93, é de c78 (6,7; 3,0; 5,0; 1,7), uma Versicolor na fronteira com as Virginica, perto das duas espécies.
- **Grafo simples.** É simples por construção: a diagonal nunca é avaliada e cada par gera no máximo uma aresta. O cabeçalho confirma isso contando laços e arestas múltiplas na própria matriz.

O grafo não usa a espécie, mas cruzar as arestas com ela depois explica por que existe um único componente. As Setosa formam um bloco quase completo e se ligam ao resto por apenas 54 arestas, todas com Versicolor.

Par de espécies	Arestas	Pares possíveis	Pares ligados
Setosa–Setosa	1.222	1.225	99,8%
Versicolor–Versicolor	1.176	1.225	96,0%
Virginica–Virginica	1.117	1.225	91,2%
Versicolor–Virginica	1.681	2.500	67,2%
Setosa–Versicolor	54	2.500	2,2%
Setosa–Virginica	0	2.500	0,0%

Espécie	Grau mínimo	Grau médio	Grau máximo
Setosa	47	50,0	53
Versicolor	46	81,7	93
Virginica	30	78,3	92

A menor DEN entre uma Setosa e uma flor de outra espécie é 0,2315 (c24 e c99, DE = 1,64 cm). Com limiar abaixo desse valor, o grafo se divide em 2 componentes: as 50 Setosa e as 100 Versicolor e Virginica. Já Versicolor e Virginica têm 67,2% dos pares ligados e não se separam com nenhum limiar razoável; em 0,1 o grafo tem 4 componentes (97, 50, 2 e 1 vértices).

7. Visualização 3D em Python

O script `visualizar_grafo_3d.py` lê o grafo persistido (`grafo_iris.csv`, não a base) e desenha os 150 vértices e as 5.250 arestas em 3D com matplotlib. Arrastar com o mouse gira o grafo e muda o ponto de vista.

- **Fonte do script:** adaptado dos exemplos oficiais do Matplotlib, 3D scatterplot (vértices) e Line3DCollection (arestas). A fonte também está citada no início do script.
- **Posição:** cada vértice fica nas coordenadas de 3 das 4 medidas: `petal.length`, `petal.width` e `sepal.length`. A quarta medida não aparece na posição, mas entra no cálculo da DE.
- **Cor:** grau do vértice, na escala `viridis` do matplotlib, do roxo (grau 30) ao amarelo (grau 93).
- **Mouse:** arrastar com o botão esquerdo gira o grafo; o botão direito ou a roda dão zoom. É o comportamento padrão dos eixos 3D do matplotlib.
- **Uso:** `python visualizar_grafo_3d.py [grafo.csv]`; sem argumento, abre `grafo_iris.csv`.

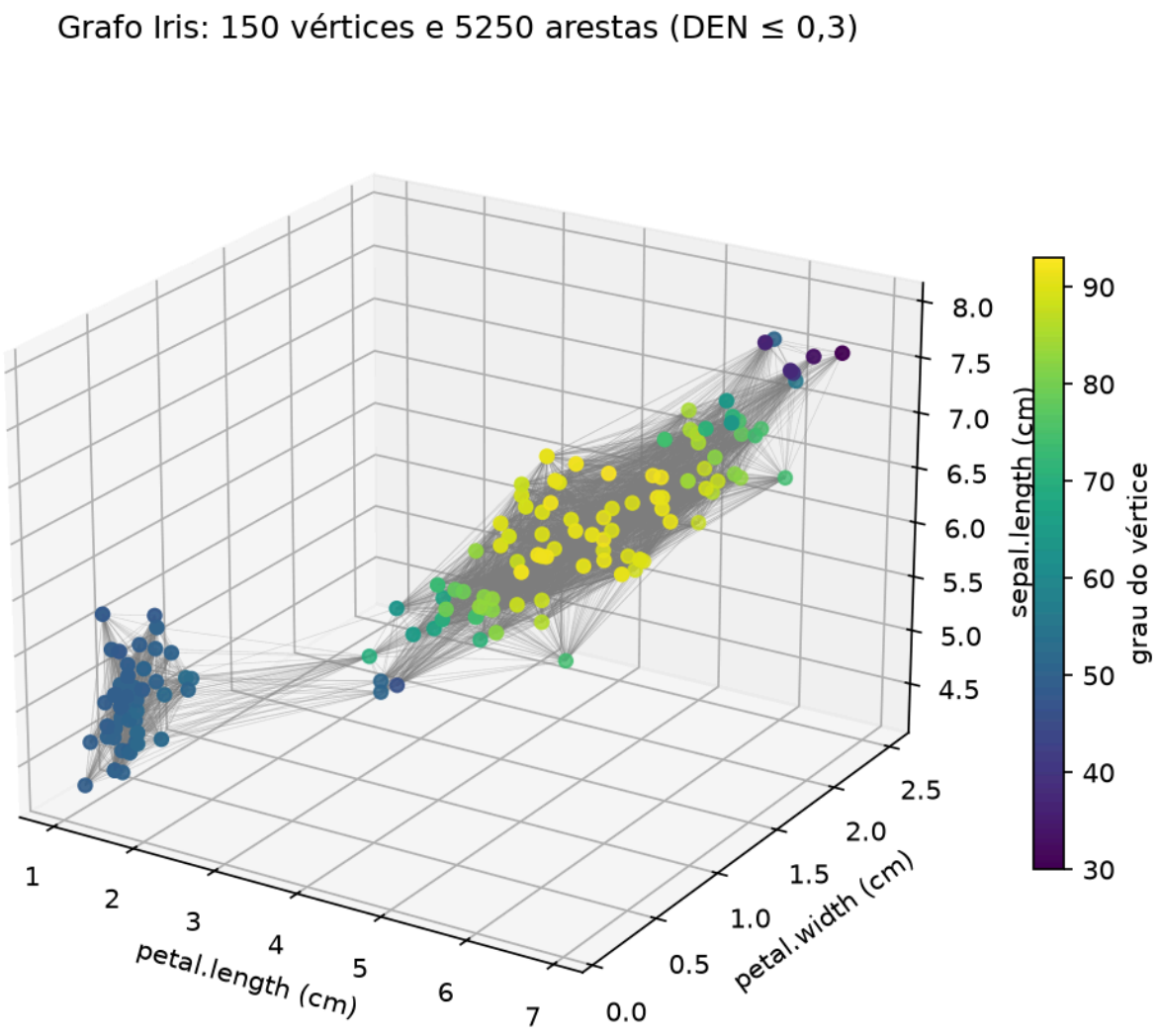


Figura 1 — Vista com elevação de 22° e azimute de −58°.

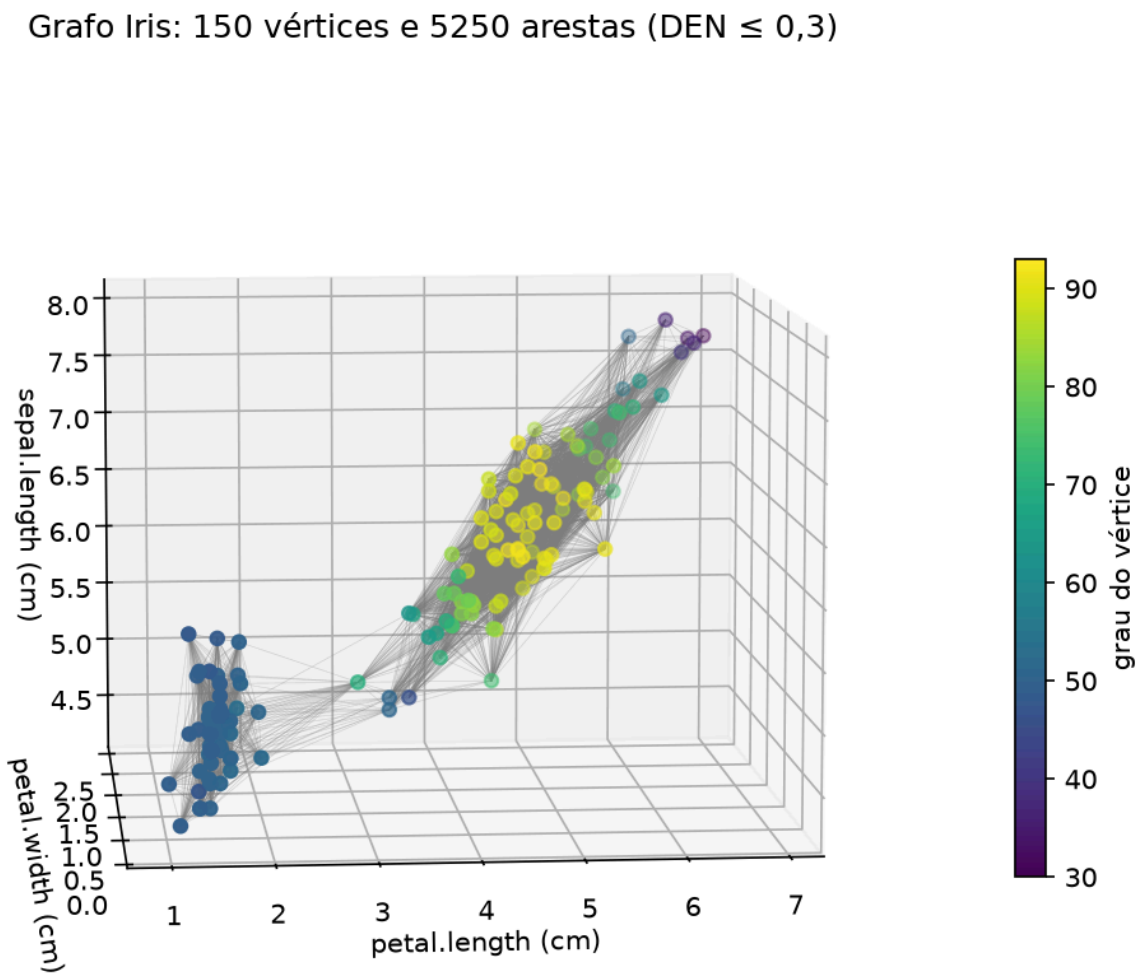


Figura 2 — Vista quase lateral (elevação de 8°, azimute de −95°): petal.length × sepal.length.

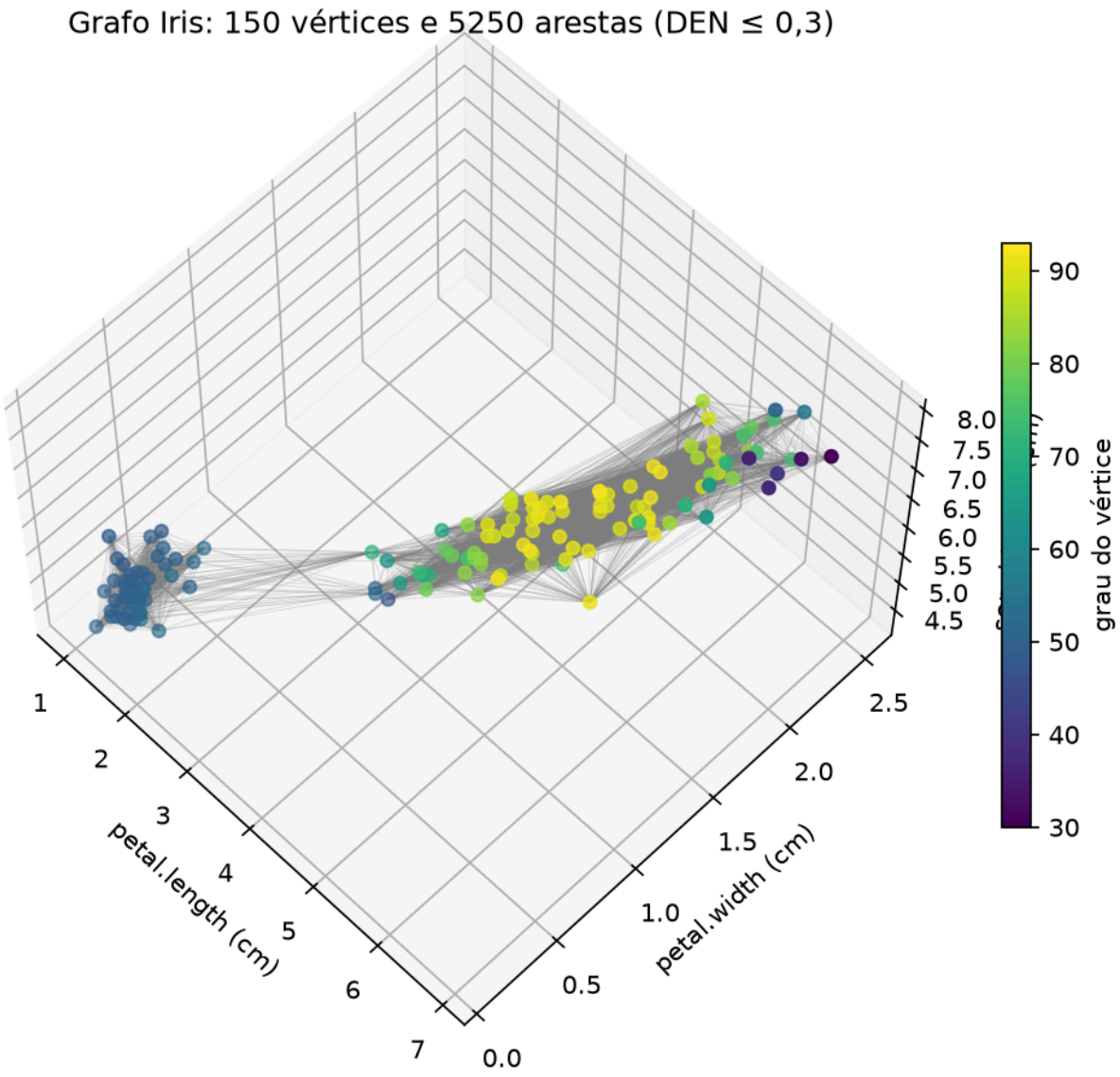


Figura 3 — Vista de cima (elevação de 60°, azimute de −45°).

As três vistas mostram o mesmo que o cabeçalho: um único componente conexo. As Setosa (à esquerda, em azul, grau perto de 50) formam um bloco compacto, ligado ao grupo Versicolor e Virginica por um feixe estreito de arestas: as 54 ligações Setosa–Versicolor. Os pontos roxos no canto superior direito são as Virginica maiores, como c119, com os menores graus do grafo.

8. Como compilar, executar e verificar

O programa foi compilado e testado com gcc 6.3 (MinGW) no Windows 11 e com gcc 13.3 no Ubuntu 24.04, e o script de visualização roda em Python 3.12 com numpy e matplotlib. Os comandos são executados dentro da pasta do trabalho.

```
# compilar (ou: gcc -std=c99 -Wall -Wextra -pedantic -O2 -o grafo_iris main.c grafo.c -lm)
mingw32-make

# carga primária: base -> DE -> DEN -> limiar -> matriz; mostra o cabeçalho e salva o grafo
grafo_iris carregar original_IrisDataset.csv grafo_iris.csv

# recarga sem a base original (o 2o arquivo é opcional: salva uma cópia)
grafo_iris recarregar grafo_iris.csv grafo_copia.csv

# visualização 3D interativa
python visualizar_grafo_3d.py grafo_iris.csv
```

A bateria de testes tem 169 casos e fica na pasta `testes/`, fora da entrega. Ela roda igual no Windows e no Linux e confere o programa contra uma reimplementação independente em numpy.

O que é verificado	Casos
Compilação com `-Werror` (qualquer aviso reprova)	1
Base Iris: itens i a viii, 5.250 arestas, matriz simétrica de 0/1, tela igual ao arquivo, recarga idêntica, recarga de arquivo com uma linha extra no cabeçalho	16
36 bases (2, 15 e 300 observações; `\r\n`; linhas em branco; espaços após as vírgulas; 30 aleatórias de 2 a 200 observações e de 1 a 4 componentes): carga, recarga idêntica e conferência com numpy	108
Formatos diferentes geram o mesmo grafo da base original; observações todas iguais; aleatórias com mais de um componente	5
Multigrafo com laço e aresta dupla, e grafo de vértices isolados, montados à mão	6
Base inválida (valor não numérico, coluna faltando, 1 observação, vazia, só cabeçalho, inexistente) e saída em pasta inexistente	8
Grafo salvo corrompido (10 tipos), inexistente e com linhas em branco	12
Uso errado da linha de comando	5
Memória: nenhum relatório de sanitizer ou valgrind e nenhum código de saída anormal	2
Visualização: pontos, arestas e título batem com o arquivo; a janela real gira com o arraste do mouse	6

Todas as execuções passaram. No Linux ficam de fora os 43 casos que dependem de Python (numpy e visualização).

Execução	Resultado
Windows 11, gcc 6.3, com numpy e janela 3D	169 de 169
Ubuntu 24.04, gcc 13.3, com AddressSanitizer e UBSan	126 de 126
Ubuntu 24.04, sob valgrind	126 de 126; toda a memória liberada

Execução	Resultado
Grafos gerados no Linux × no Windows	91 arquivos idênticos
Conteúdo do zip de entrega, extraído numa pasta limpa	168 de 168 (sem a janela 3D)

Para saber se a bateria pega erros de verdade, 11 bugs foram plantados no código, um de cada vez: limiar errado, tabela sem normalizar, coluna da espécie lida como medida, laço contando 1 no grau, arestas sem contar laços, arestas múltiplas ignoradas, empate resolvido pelo último par, tamanho de componente errado, recarga sem validar vértices, vazamento de memória e leitura fora do vetor. A bateria falhou com todos os 11; os dois de memória foram pegos pelo AddressSanitizer no Linux.

9. Conclusão

Os seis requisitos funcionais foram implementados e verificados. O grafo resultante tem 150 vértices, 5.250 arestas e é simples e conexo.

Requisito	Onde é atendido
a) Matriz de adjacências	Campo `adj` da struct `grafo` (seção 4)
b) Carga primária ignorando a 1ª coluna	`grafo_carregar_base`, com `%*[^,]` no `sscanf` (seção 2)
c) Vértices c1 a c150	Uma observação por vértice, na ordem do arquivo (seção 2)
d) Aresta se DEN ≤ 0,3	Passos 1 a 4 da carga primária (seção 3)
e) Persistência com cabeçalho i–viii e recarga	`grafo_salvar` e `grafo_carregar_salvo` (seção 5)
f) Visualização 3D com mouse e fonte citada	`visualizar_grafo_3d.py` (seção 7)

Com o limiar 0,3, o grafo junta as três espécies em um só componente, porque 54 pares Setosa–Versicolor ficam a menos de 2,13 cm. Um limiar abaixo de 0,2315 isolaria as Setosa, enquanto Versicolor e Virginica continuariam misturadas, o que bate com a sobreposição conhecida dessas duas espécies na base Iris.

10. Referências

- Especificação da Tarefa 1_A, TEG (PDF disponibilizado no Moodle).
- UCI Machine Learning Repository, Iris, indicada na especificação.
- Wikipédia, Conjunto de dados flor Iris, indicada na especificação.
- Meu Guru, Distâncias: as 3 mais usadas, indicada na especificação.
- Towards Data Science, Normalization vs Standardization Explained, indicada na especificação.
- Matplotlib, 3D scatterplot, base do script de visualização.
- Matplotlib, mpl_toolkits.mplot3d.art3d.Line3DCollection, usada para desenhar as arestas.